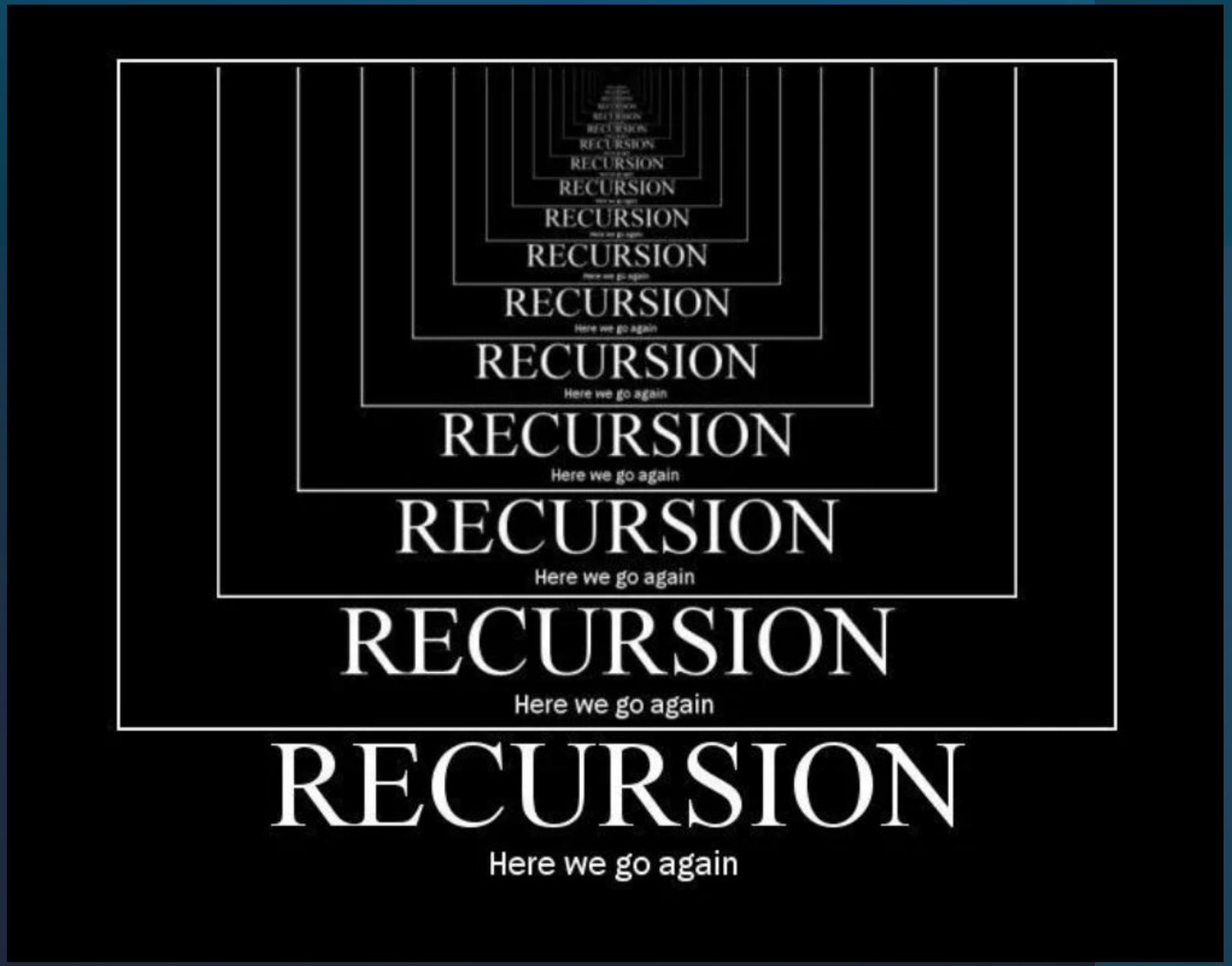
Data Structures and Algorithms I.



University of Szeged

Robert Nagy

2025/26. Spring Semester


RECURSION
Here we go again
RECURSION
Here we go again
RECURSION
Here we go again
RECURSION
Here we go again
RECURSION
Here we go again
RECURSION
Here we go again
RECURSION
Here we go again
RECURSION
Here we go again
RECURSION
Here we go again

main.py

```
1
2 def factorial(n):
3     if n <= 1:
4         return 1
5     return n * factorial(n - 1)
6
```

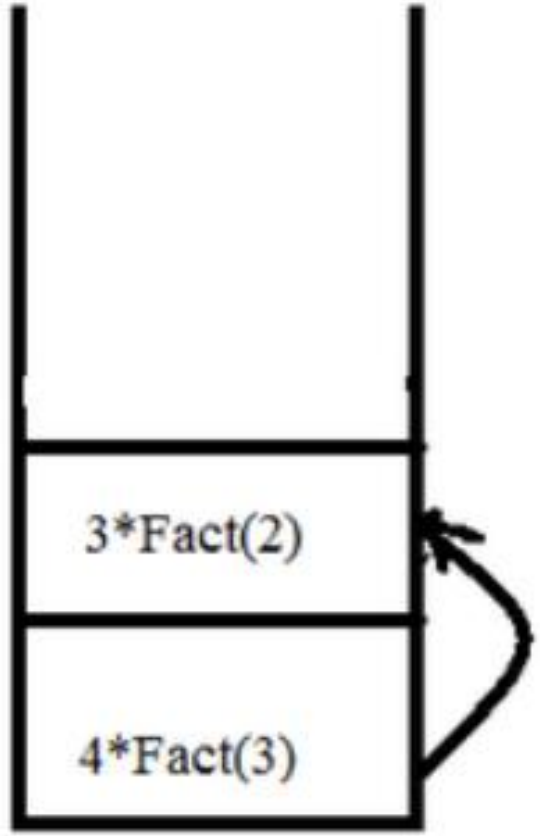


After the first call

https://stackoverflow.com/questions/19865503/can-recursion-be-named-as-a-simple-function-call



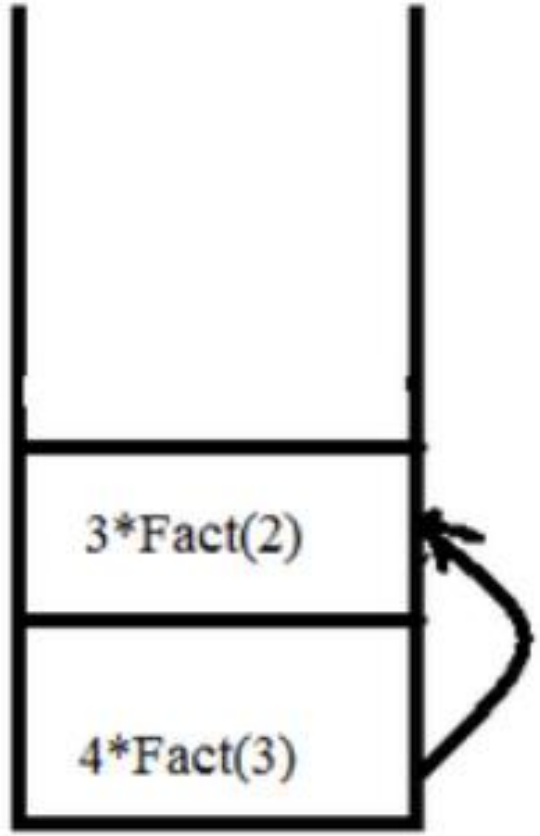
After the first call



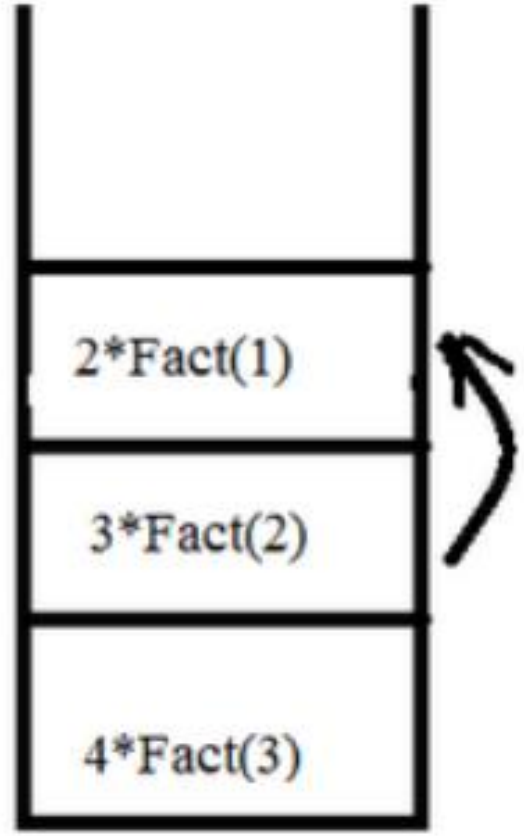
second call



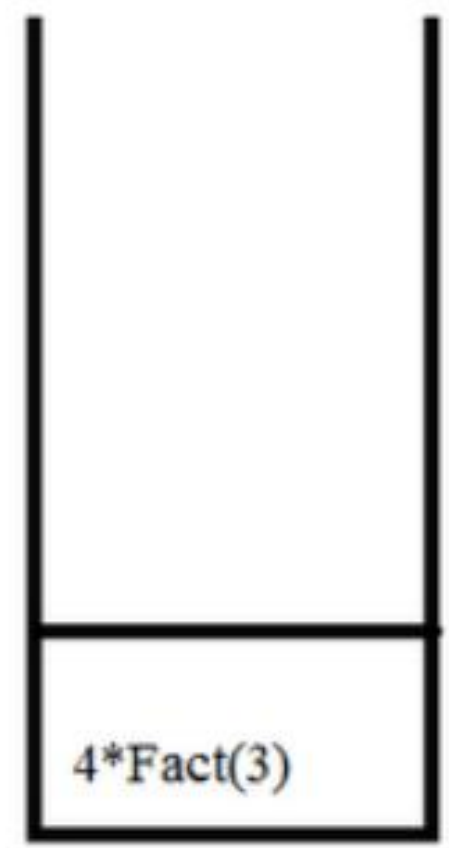
After the first call



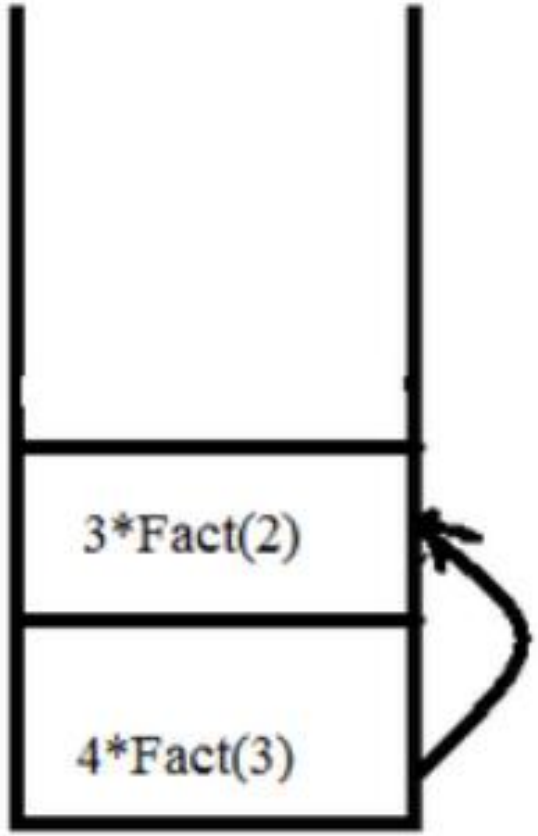
second call



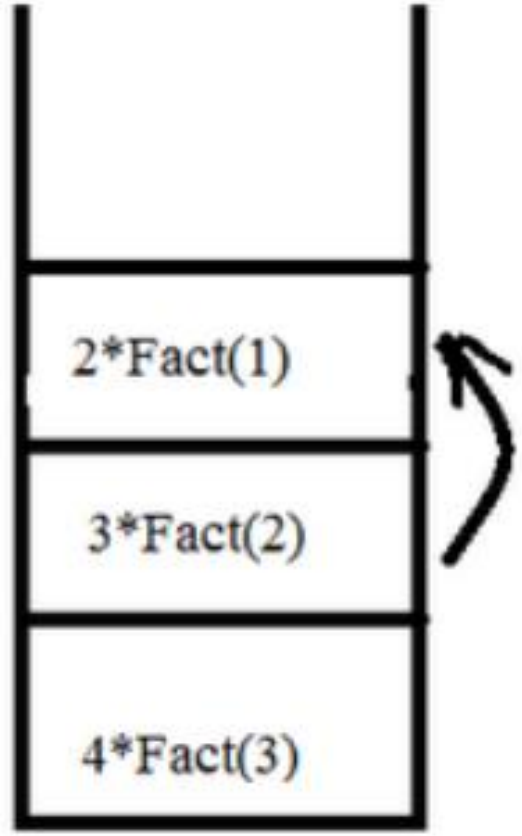
third call



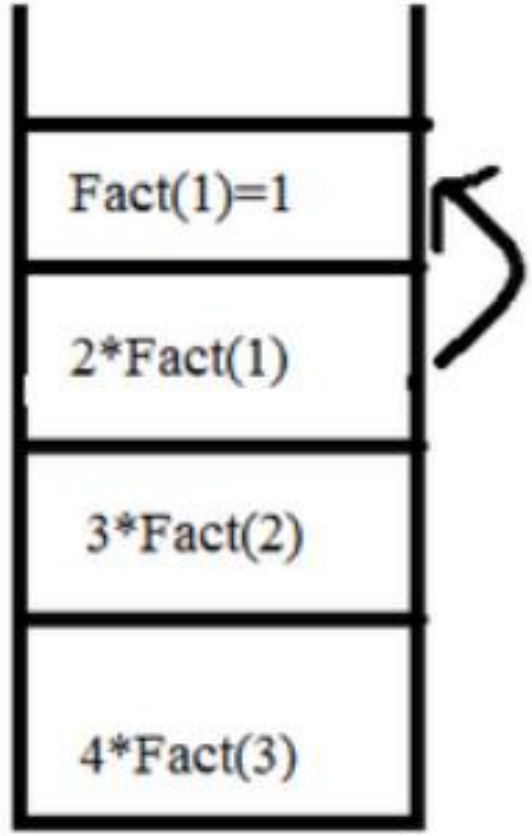
After the first call



second call

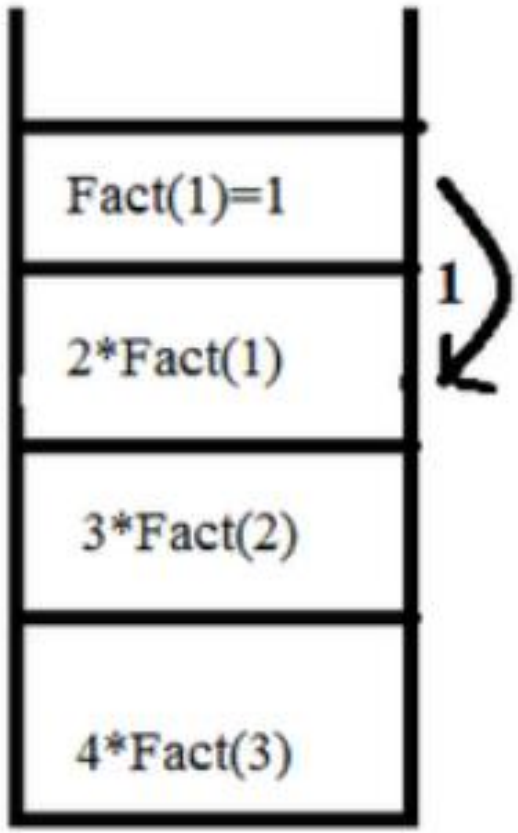


third call

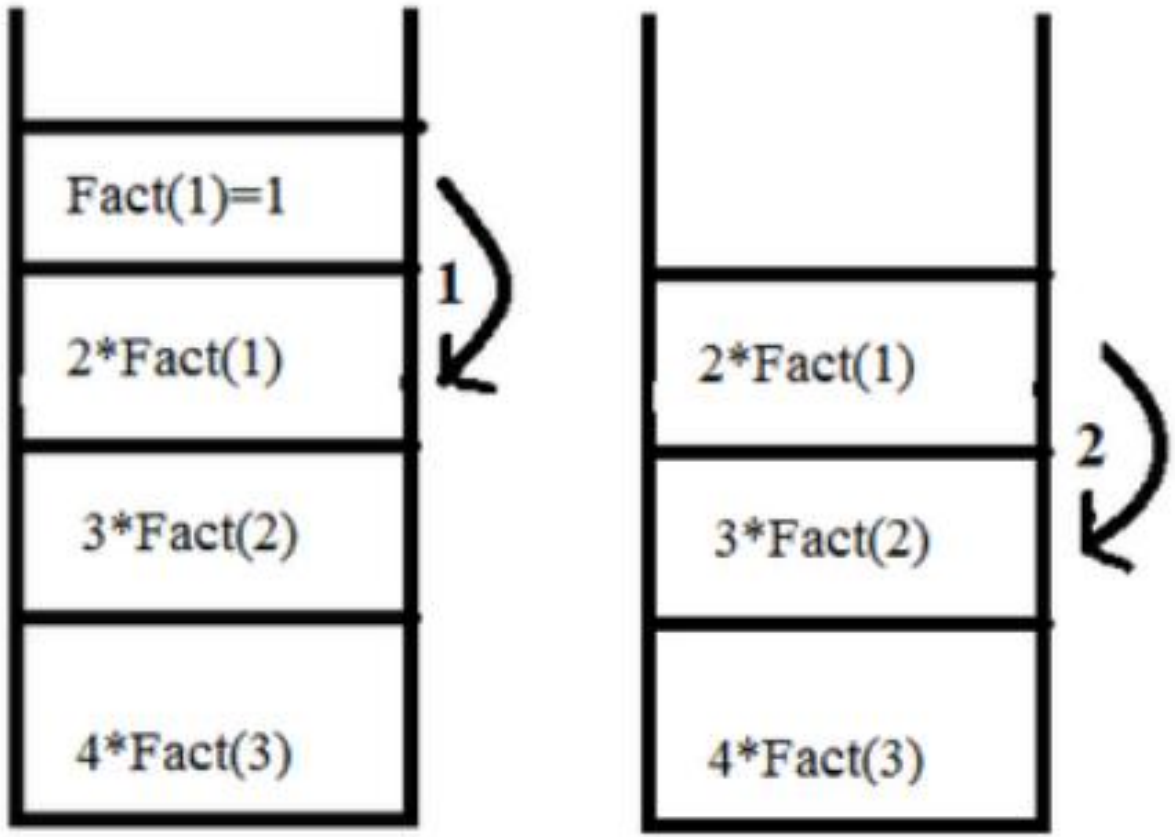


fourth call

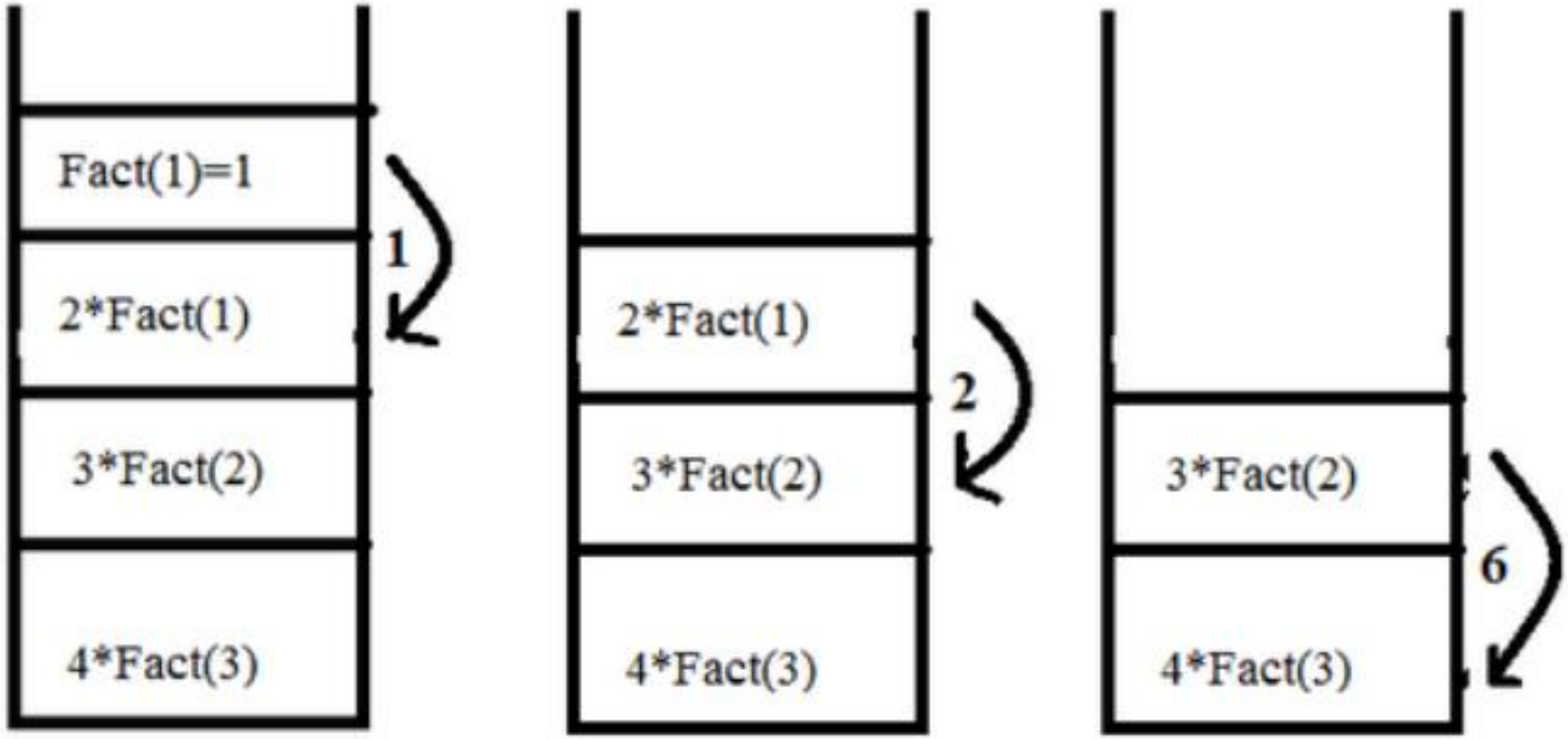
Returning values from base case to caller function



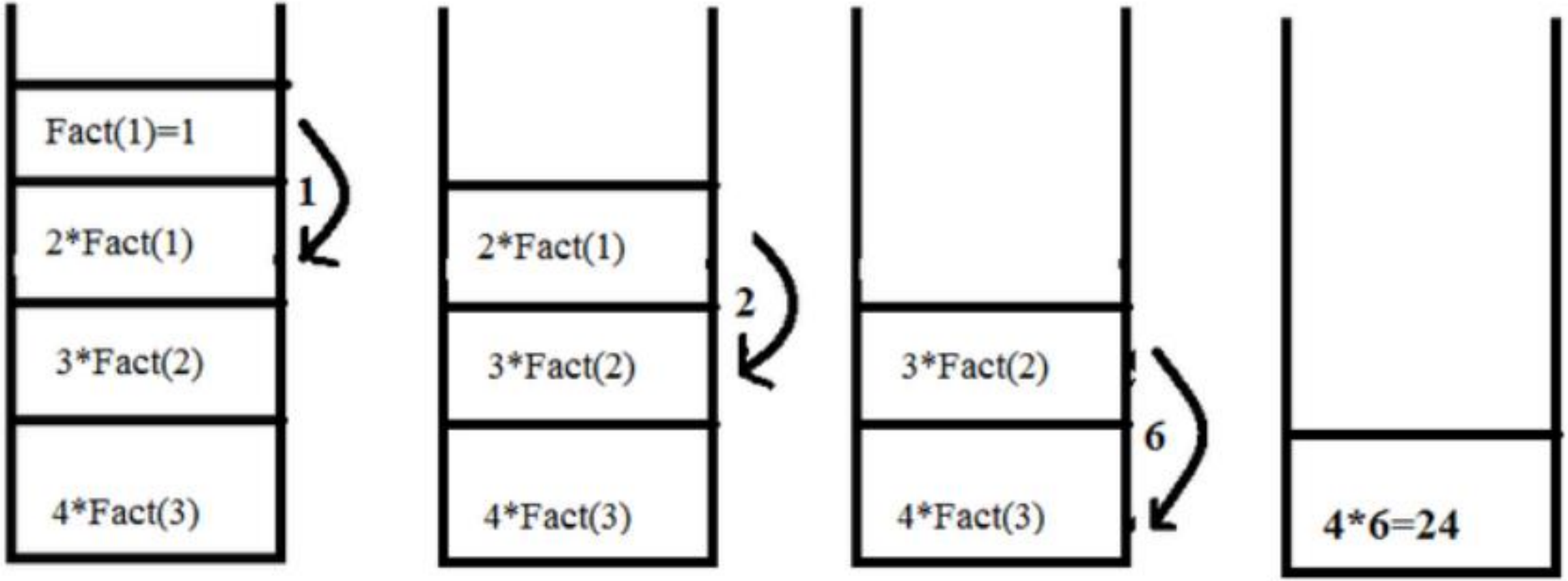
Returning values from base case to caller function



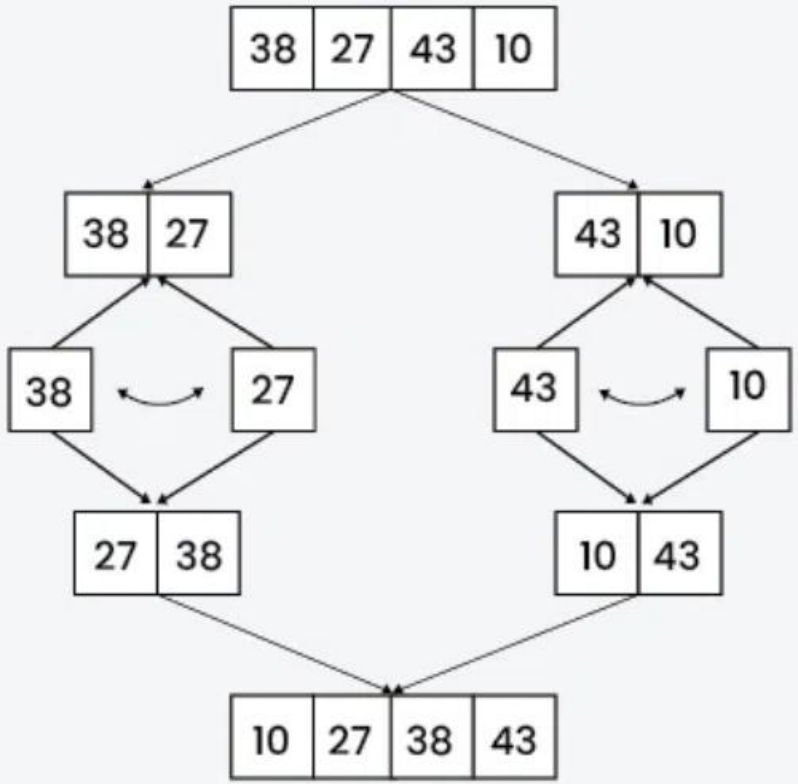
Returning values from base case to caller function



Returning values from base case to caller function



Divide and Conquer



```
MERGE-SORT(A, p, r)
1  if p ≥ r                                   // zero or one element?
2      return
3  q = ⌊(p + r)/2⌋                            // midpoint of A[p : r]
4  MERGE-SORT(A, p, q)                        // recursively sort A[p : q]
5  MERGE-SORT(A, q + 1, r)                    // recursively sort A[q + 1 : r]
6  // Merge A[p : q] and A[q + 1 : r] into A[p : r].
7  MERGE(A, p, q, r)
```

Merge sort – time complexity analysis

- Divide: $\Theta(1)$

Merge sort – time complexity analysis

- Divide: $\Theta(1)$
- Conquer: $2\mathrm{T}(n/2)$

Merge sort – time complexity analysis

- Divide: $\Theta(1)$
- Conquer: $2\mathrm{T}(n/2)$
- Combine: : $\Theta(n)$

$$T(n) = \Theta(1) + 2T\left(\frac{n}{2}\right) + \Theta(n)$$

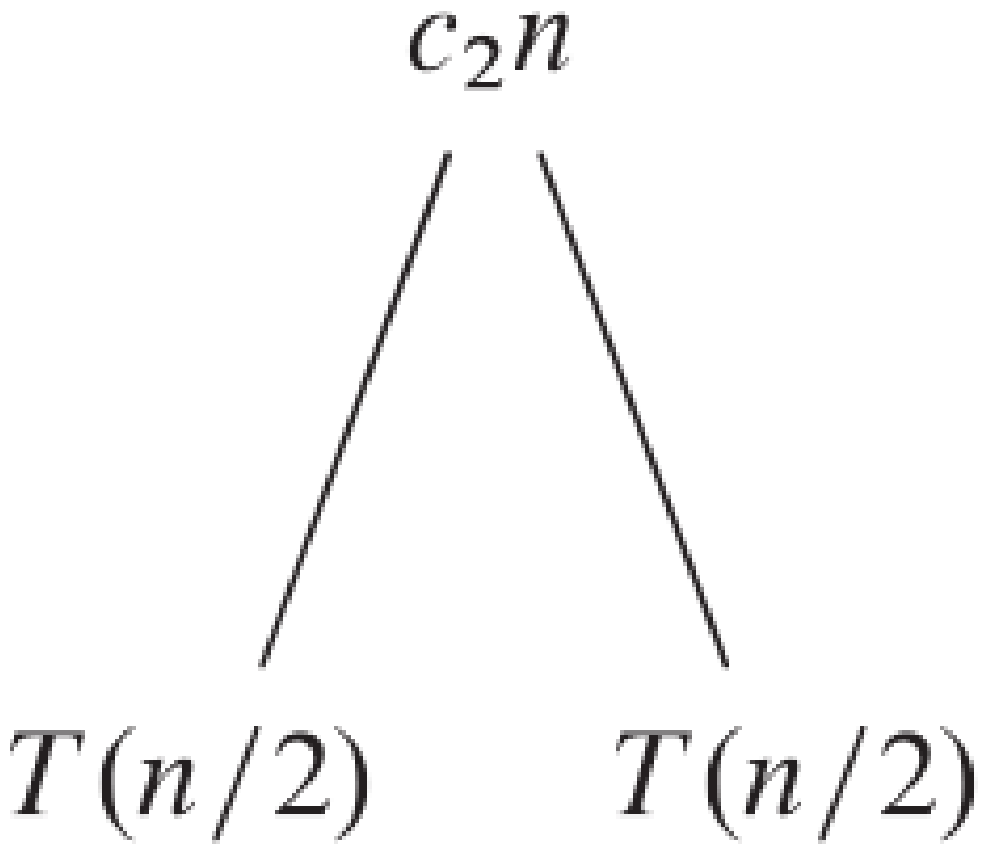
The time complexity of Merge Sort

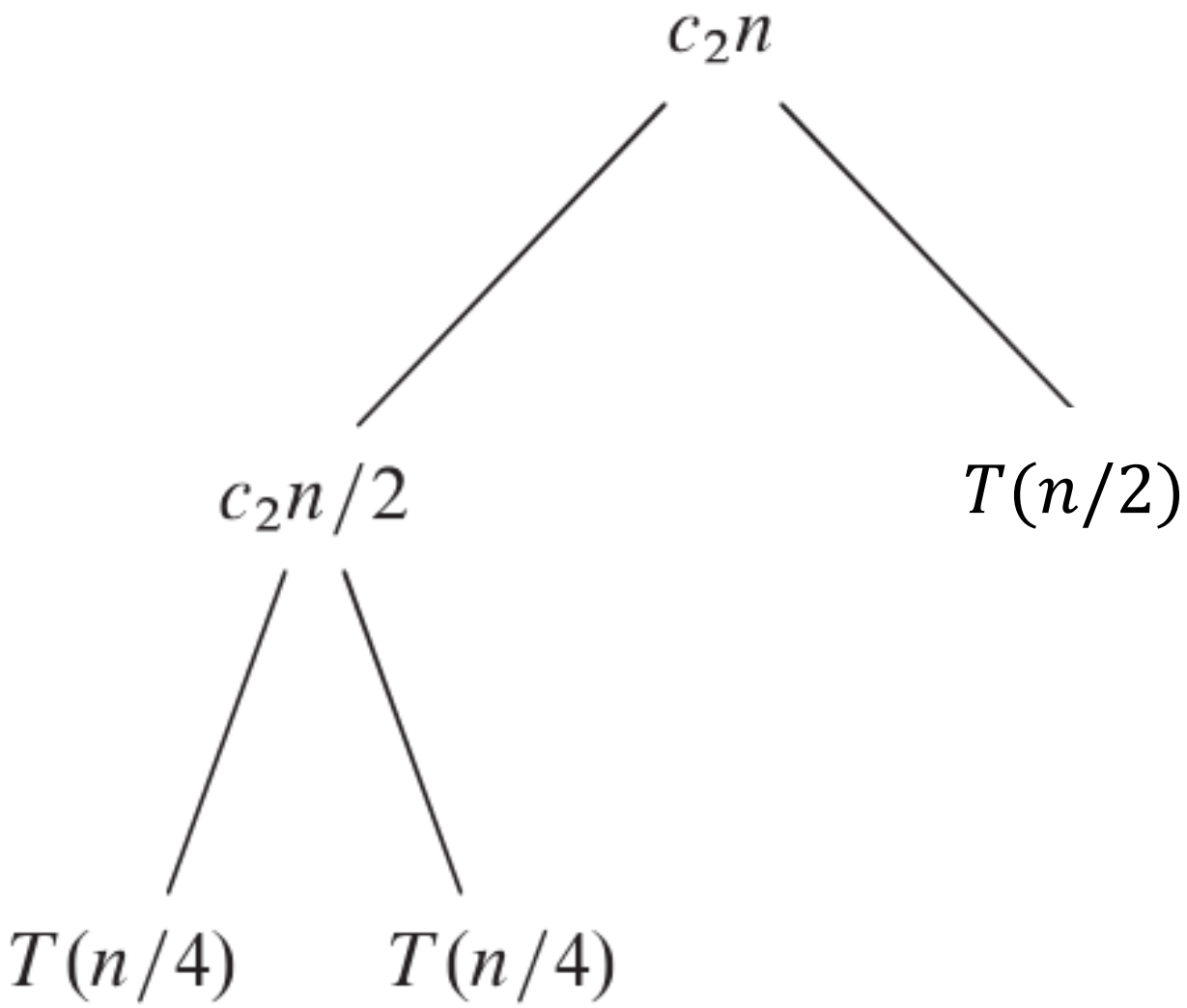
$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

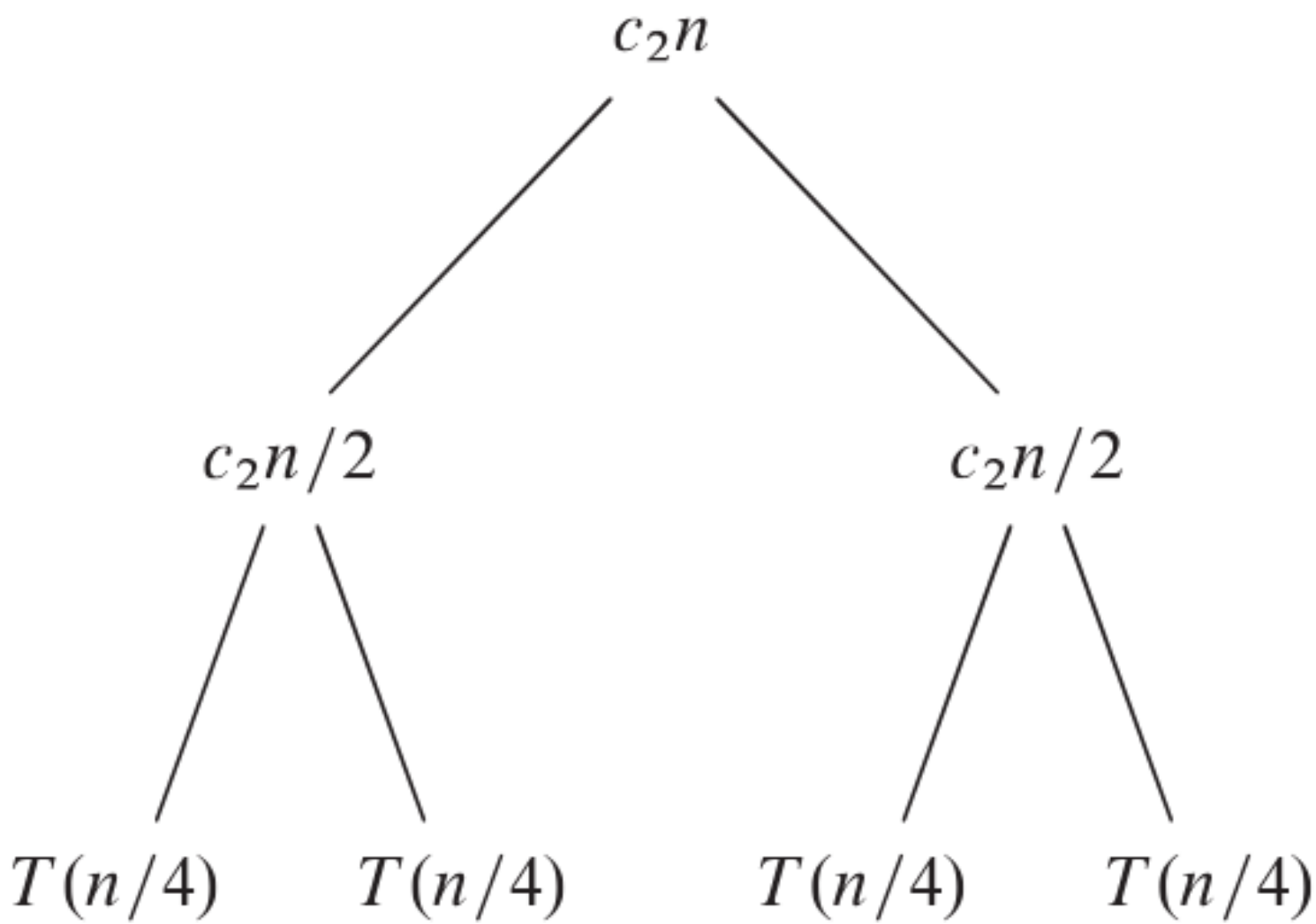
The time complexity of Merge Sort

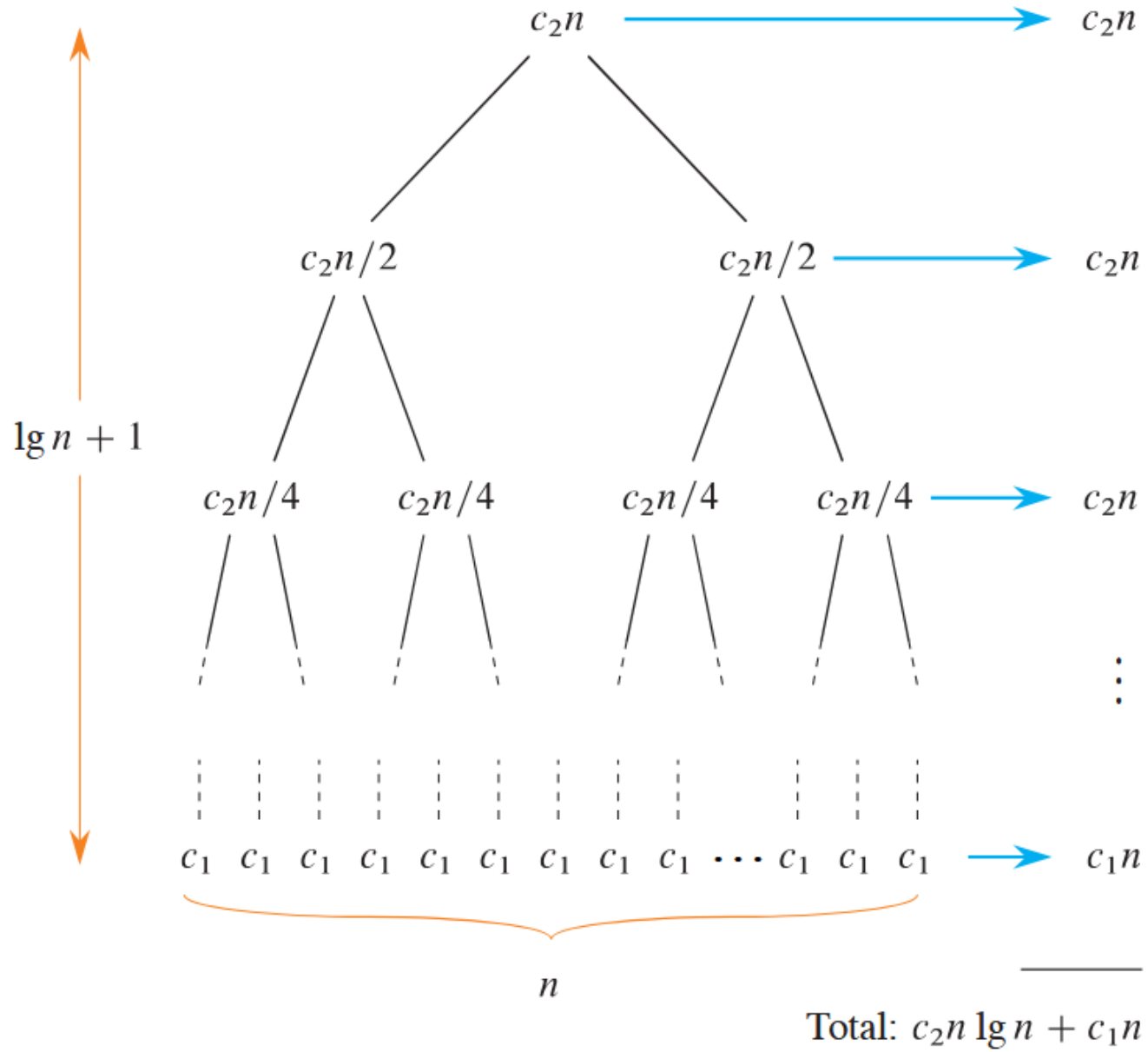

It’s drawing time!

Recursion tree




c_2n
$c_2n/2$
$T(n/2)$
$T(n/4)$
$T(n/4)$


c_2n
$c_2n/2$
$c_2n/2$
$T(n/4)$
$T(n/4)$
$T(n/4)$
$T(n/4)$


c_2n
c_2n
$c_2n/2$
$c_2n/2$
c_2n
$\lg n + 1$
$c_2n/4$
$c_2n/4$
$c_2n/4$
$c_2n/4$
c_2n
c_1 c_1 c_1 c_1 c_1 c_1 c_1 c_1 c_1 $\cdots$ c_1 c_1 c_1
c_1n
n
Total: $c_2n \lg n + c_1n$

Exercises

For each of the following recurrences, sketch its recursion tree, and guess a good asymptotic upper bound on its solution.

a. $T(n) = T(n/2) + n^3$.

b. $T(n) = 4T(n/3) + n$.

c. $T(n) = 4T(n/2) + n$.

d. $T(n) = 3T(n-1) + 1$.

Master theorem (method)

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- $a > 0 \; and \; b > 1$

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- $a > 0 \; and \; b > 1$
- $f(n)$ is called the “driving function”
  - $f(n) = divide + combine$

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- $a > 0 \; and \; b > 1$
- $f(n)$ is called the “driving function”
  - $f(n) = divide + combine$
- $n^{log_b(a)}$ is called the “watershed function”

3 Special cases

Case 1

$if\ \exists \epsilon > 0,$

Case 1

$$if\ \exists \epsilon > 0, such\ that\ f(n) = O\left(n^{log_b(a)-\epsilon}\right),$$

Case 1

$$if\ \exists \epsilon > 0, such\ that\ f(n) = O\left(n^{log_b(a)-\epsilon}\right), then\ T(n) = \Theta(n^{log_b(a)})$$

Example

$$T(n) = 9T\left(\frac{n}{3}\right) + n$$

$$T(n) = 9T\left(\frac{n}{3}\right) + n$$

- $a = 9\ and\ b = 3$

$$T(n) = 9T\left(\frac{n}{3}\right) + n$$

- $a = 9\ and\ b = 3$
- $n^{log_b(a)} = n^{log_3(9)} = n^2 = \Theta(n^2)$

$$T(n) = 9T\left(\frac{n}{3}\right) + n$$

- $a = 9\ and\ b = 3$
- $n^{log_b(a)} = n^{log_3(9)} = n^2 = \Theta(n^2)$
- $f(n) = n = O(n^{2-\epsilon}), \forall \epsilon \leq 1$

$$T(n) = 9T\left(\frac{n}{3}\right) + n$$

- $a = 9\ and\ b = 3$
- $n^{log_b(a)} = n^{log_3(9)} = n^2 = \Theta(n^2)$
- $f(n) = n = O(n^{2-\epsilon}), \forall \epsilon \leq 1$

Therefore, $T(n) = \Theta(n^2)$

Case 2

$if\ \exists k \geq 0$

Case 2

$$if\ \exists k \geq 0, such\ that\ f(n) = \Theta\left(n^{log_b(a)} * lg^k(n)\right)$$

Case 2

$$if\ \exists k \geq 0, such\ that\ f(n) = \Theta\left(n^{log_b(a)} * lg^k(n)\right),$$
$$then\ T(n) = \Theta\left(n^{log_b(a)} * lg^{k+1}(n)\right)$$

Example

$$T(n) = T\left(\frac{2n}{3}\right) + 1$$

- $a = 1\ and\ b = \frac{3}{2}$

$$T(n) = T\left(\frac{2n}{3}\right) + 1$$

- $a = 1\ and\ b = \frac{3}{2}$
- $n^{log_b(a)} = n^{log_{3/2}(1)} = n^0 = 1 = \Theta(1)$

$$T(n) = T\left(\frac{2n}{3}\right) + 1$$

- $a = 1\ and\ b = \frac{3}{2}$
- $n^{log_b(a)} = n^{log_{3/2}(1)} = n^0 = 1 = \Theta(1)$
- $f(n) = 1 = \Theta\left(n^{log_b(a)} * lg^0(n)\right) = \Theta(1)$

Therefore, $T(n) = \Theta(lg(n))$

$$T(n) = 2T\left(\frac{n}{2}\right) + n * lg(n)$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n * lg(n)$$

Next, let's look at the recurrence $T(n) = 2T(n/2) + n \lg n$, where we have $a = 2, b = 2$, and $n^{\log_b a} = n^{\log_2 2} = n$. Case 2 applies since $f(n) = n \lg n = \Theta(n^{\log_b a} \lg^1 n)$. We conclude that the solution is $T(n) = \Theta(n \lg^2 n)$.

Case 3

$$if\ \exists \epsilon > 0, such\ that\ f(n) = \Omega\left(n^{log_b(a)+\epsilon}\right)$$

Case 3

$$if\ \exists \epsilon > 0, such\ that\ f(n) = \Omega\left(n^{log_b(a)+\epsilon}\right)$$

$$and\ if\ \exists c < 1, such\ that\ af\left(\frac{n}{b}\right) \leq cf(n), \forall n > n_0$$

Case 3

$$if\ \exists \epsilon > 0, such\ that\ f(n) = \Omega\left(n^{log_b(a)+\epsilon}\right)$$

$$and\ if\ \exists c < 1, such\ that\ af\left(\frac{n}{b}\right) \leq cf(n), \forall n > n_0$$

$$then\ T(n) = \Theta(f(n))$$

Example

$$T(n) = 3T\left(\frac{n}{4}\right) + n * lg(n)$$

- $n^{log_b(a)} = n^{log_4(3)} = O(n^{0.793})$

$$T(n) = 3T\left(\frac{n}{4}\right) + n * lg(n)$$

- $n^{log_b(a)} = n^{log_4(3)} = O(n^{0.793})$
- $f(n) = n * lg(n) = \Omega(n^{0.793+\epsilon})$
  - $\epsilon \approx 0.2$

$$T(n) = 3T\left(\frac{n}{4}\right) + n * lg(n)$$

- $n^{log_b(a)} = n^{log_4(3)} = O(n^{0.793})$
- $f(n) = n * lg(n) = \Omega(n^{0.793+\epsilon})$
  - $\epsilon \approx 0.2$
- $3\left(\frac{n}{4}\right)\left(lg\left(\frac{n}{4}\right)\right) \leq \frac{3}{4}n * lg(n)$

Therefore, $T(n) = \Theta(n * lg(n))$

Master method - recap

- If the recurrence is in the form $T(n) = aT\left(\frac{n}{b}\right) + f(n)$
- $f(n)$ is called the "driving function"
- $n^{log_b(a)}$ is called the "watershed function"
- 3 Special cases make calculations easier:
  1. If $n^{log_b(a)} \gg f(n), then\ T(n) = \Theta\left(n^{log_b(a)}\right)$
  2. If $n^{log_b(a)} \approx f(n), then\ T(n) = \Theta\left(n^{log_b(a)} * lg^{k+1}(n)\right)$
  3. If $n^{log_b(a)} \ll f(n), then\ T(n) = \Theta\left(f(n)\right)$

Appendix

- When we write "$\mathrm{g(n)} \gg f(n)$" In the context of the"Master Theorem", we mean that *g* grows *polynomially faster than f*
  - *Which means* $\frac{g(n)}{f(n)} = n^k,\ k > 0$
- *Examples:*
  - $\frac{n^2}{n^{1.99}} = n^{0.01}$, therefore $n^2 \gg n^{1.99}$
  - $\frac{n*lg(n)}{n} = lg(n)$, therefore $n \approx n * lg(n)$

Exercises

Use the master method to give tight asymptotic bounds for the following recurrences.

a. $T(n) = 2T(n/4) + 1$.

b. $T(n) = 2T(n/4) + \sqrt{n}$.

c. $T(n) = 2T(n/4) + \sqrt{n}\lg^2 n$.

d. $T(n) = 2T(n/4) + n$.

e. $T(n) = 2T(n/4) + n^2$.